

Competitive Exam Practice 1 — Answer Key

Foundations, Pointers & ADTs

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 12, 2026

- Q1 (C).** P1 returns the address of its local variable `x` (a dangling pointer — the stack frame dies when `g` returns), P2 dereferences the uninitialized pointer `px` before any `malloc`, while P3 requests a heap block first and is safe; hence only P1 and P2 cause problems.
- Q2 (D).** `pi == &i`, so `scanf` writes the entered value into `i` through the pointer; the printout is `i + 5`, exactly 5 more than the integer value entered, so the program compiles, runs, and prints “more than the integer value entered.”
- Q3 (D).** The first block from `malloc` is overwritten when `x = (int *)malloc(...)` executes inside the `if` — the old address is lost and can never be freed, so the code compiles but leaks memory; the final `free(x)` only releases the second block, and no dangling pointer is ever used.
- Q4 25.** `z = &a` makes `z` point at `a`; inside `foo`, `*p = x` writes the value of `b` (25) into `a` through the pointer, so the printed `a` is 25.
- Q5 (B).** A local variable declared inside a function is automatic storage on the **stack**; `malloc` requests bytes from the **heap** at run time.
- Q6 TRUE.** A local variable lives in its function call’s stack frame and dies on return, while a heap block requested by `malloc` remains until the program releases it with `free` — the heap’s whole point is size and lifetime decided at run time.
- Q7 TRUE.** `NULL` is the agreed “points to nothing” pointer value, and reading through it is undefined behaviour — in practice, a crash.
- Q8 (B).** `malloc(n)` asks for `n` bytes and returns their address, or `NULL` if the request cannot be met — which is why the result is always checked against `NULL` before use.
- Q9 16.** `data` occupies 4 bytes (aligned at offset 0), `next` needs 8-byte alignment so 4 padding bytes follow `data`, and the pointer occupies bytes 8–15; total $4 + 4 + 8 = 16$ bytes, which is also a multiple of the 8-byte alignment of the struct.
- Q10 (B).** The bag’s contract names the operations (`insert`, `member`, `size`) — a set of values and a set of operations — and deliberately withholds the storage scheme; “dynamic array” and “linked list” are two interchangeable *implementations* of that same contract, and code and costs are implementation details, not contract.